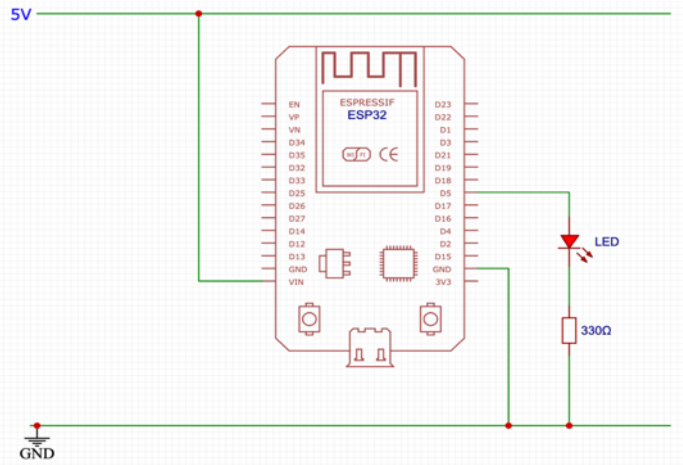
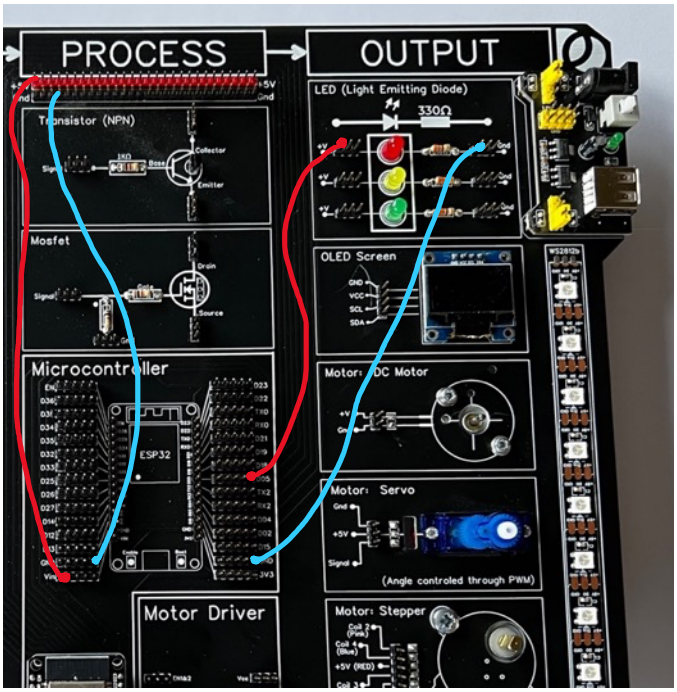
Using LEDs

We're now going to try lighting an LED again but this time though we are going to have the light come on in response to a program we have uploaded.

Construct this circuit.



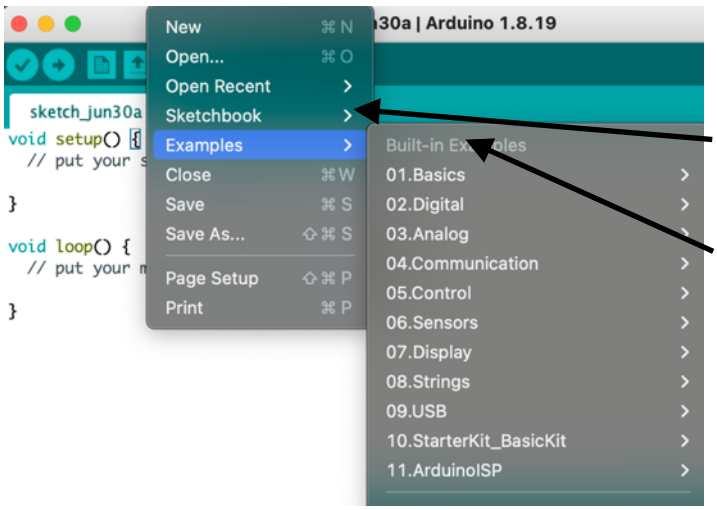


You should notice that the positive end of the LED is now attached to pin 5, rather than +V. That's because the microcontroller itself is going to be supplying the power (voltage and current). As the microcontroller is supplying the power it means we can control when it illuminates and for how long.

This circuit also still shows a connection to the voltage rails, this isn't strictly necessary in this kind of simple circuit. However, it is good practice as it makes more complex circuits easier to construct later.

If that all looks good, then let's add our first programme.

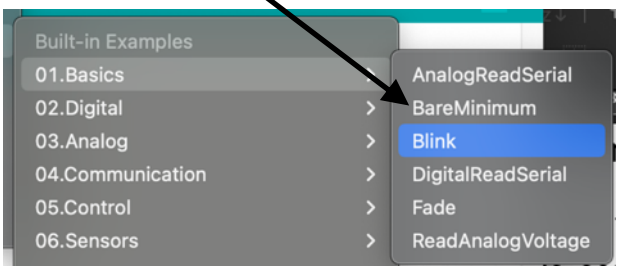
Finding and using built-in examples



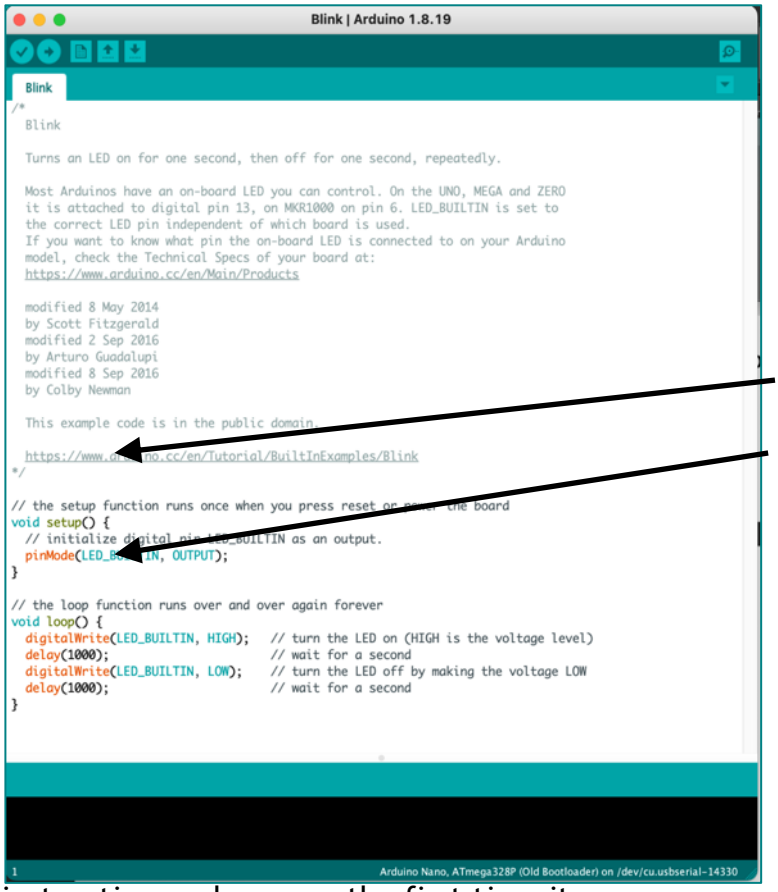
We're going to use one of the Arduino IDE's built-in examples to help us make a start.

In your Arduino IDE choose ***File*** from the menu bar and select ***Examples.***

This will open another menu, select ***01. Basics*** and then click on ***Blink.***



Selecting Blink will open a new window which has some code already written on it. Our next stage is to edit it, so it works with the circuit we've made.



Editing code

Before we start changing things let's quickly get a feel for the key parts of the code. If you look through the code, you will notice these two lines:

Void setup() {

Void loop() {

In basic Arduino programmes, **void setup ()** is a function used at the beginning of your code to set up all the things the microcontroller will need to know, such as telling it whether there is an output on pin D5 (As in our circuit), or whether there is a button (Input) attached to a different pin.

The microcontroller runs through these instructions only once – the first time it runs.

The function **Void loop() {** on the other hand starts after void setup and is an ongoing loop; this means that when it gets to the last command it cycles back and starts at the beginning again.

The only other part to talk about is the curly brackets **{ }** these enclose all the commands which belong to a particular function, a little like a fence. You will see after both of our void commands there is an opening curly bracket and if you look down a few lines there is a closing one.

While we're not going to spend a lot of time on the code let's establish the most important, unbreakable rule of curly brackets:

For every opening bracket in a programme there <u>must</u> be a closing one.

Odd numbers are not allowed and will give you all manner of headaches.

Finally let's edit some code.

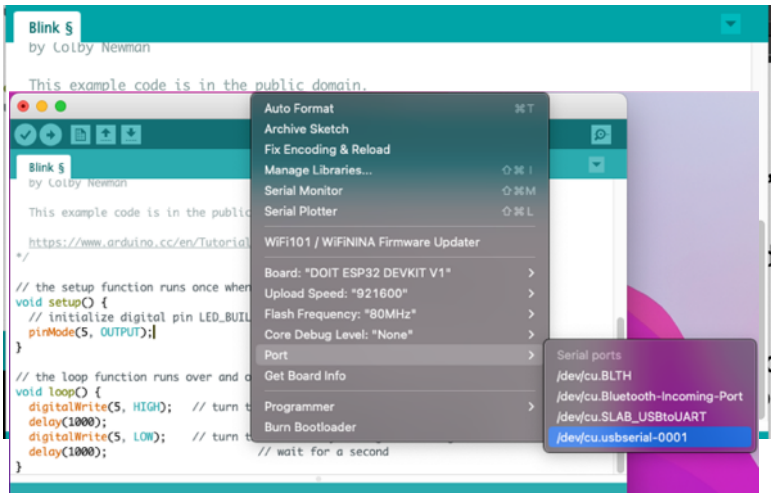
The kind people at Arduino headquarters have decided to help us out a little in our task by labelling what the lines do.

The LED_BUILTIN command works great on something like an Arduino Nano, but it doesn't work on an ESP32. We need to change the command to show the pin our LED is attached to.

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

To make our programme work you are going to remove the LED_BUILTIN command and simply replace it with '**5**' as this is the number pin on the ESP32 we've connected the LED to.



If everything has gone to plan you should have code that looks like the image on the right in your Arduino IDE.

While changing the code, you will have altered this command:

digitalWrite(5, HIGH);

We'll come on to what analogue and digital signals are later, but this command tells it whether to send power to pin 5. HIGH is high voltage (on), LOW means low voltage (off).

The last command here is **delay (1000);** This command causes a programme to pause for a set amount of time which, in this case, gives us time to see the LED change.

Delay(1000); simply means the programme won't move on to the next command for 1 second (delay's time is written in milliseconds)

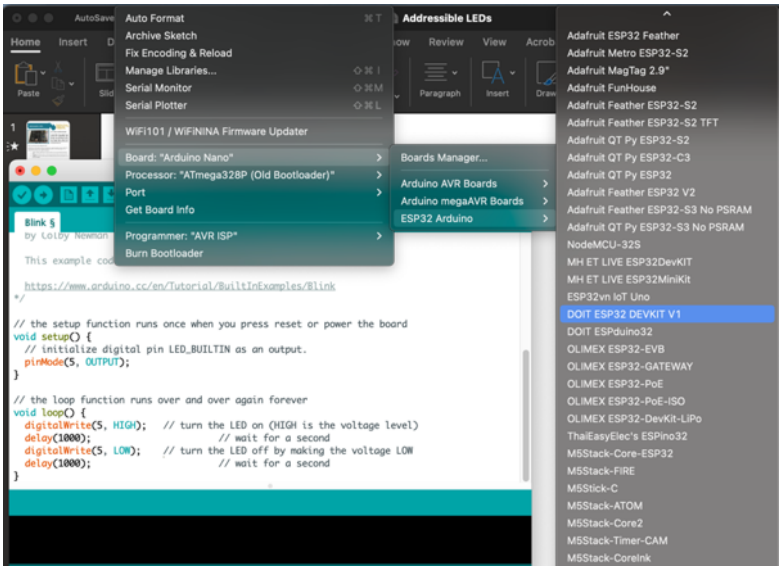
Let's upload

We need to select the microcontroller and choose a port before we can upload. First open up the tools menu and select ***board.***

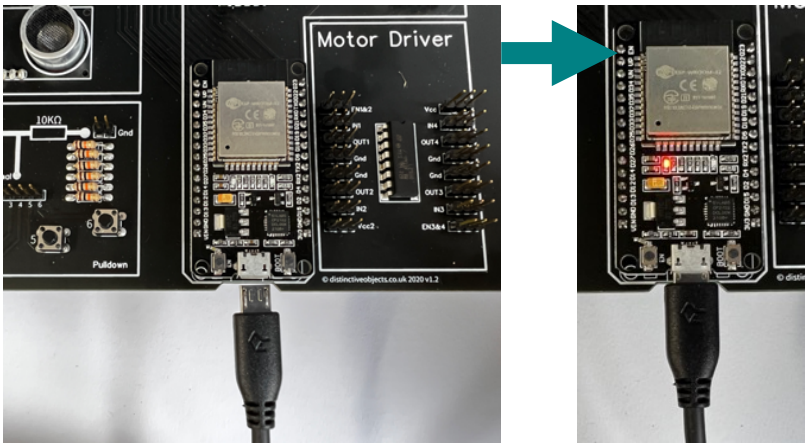
Choose ESP32 Arduino:

The next part will need you to scroll down the list a little. It is a little like a 'Where's Wally?' book in finding the right one but persevere! You are looking for this board!

DOIT ESP32 DEVKIT



Now physically connect the ESP32. The microcontroller USB is at the bottom of the board. We use a micro-USB lead to connect the computer.



Once connected the red power light on the ESP32 should light up.

Port Selection

Finally we select the port the microcontroller is attached to so they computer can talk to it.

Re-open the ***Tools*** menu and open the ***Port*** tab.

If there is only one option available, select it. On a Mac (shown) it will show up as 'usbserial-0001', on a PC it will show as COM3 (or a different number).

If you have lots of different options available, make a note of them all, close the port tab and disconnect the USB cable. When you re-open the tab there will be one missing from the list, this is the port your ESP32 is using. Reconnect the USB lead and choose that option from the port menu. (Note: if you have lots of USB ports on your computer the number in the Arduino will change depending on which USB socket you have plugged into).

Upload

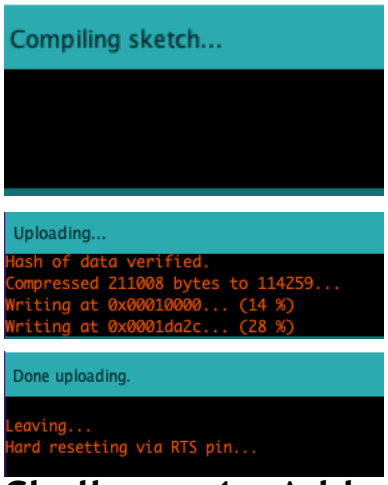
We're ready to try it out! Up at top of your IDE window are two icons, one is a tick, and the other is a right facing arrow.

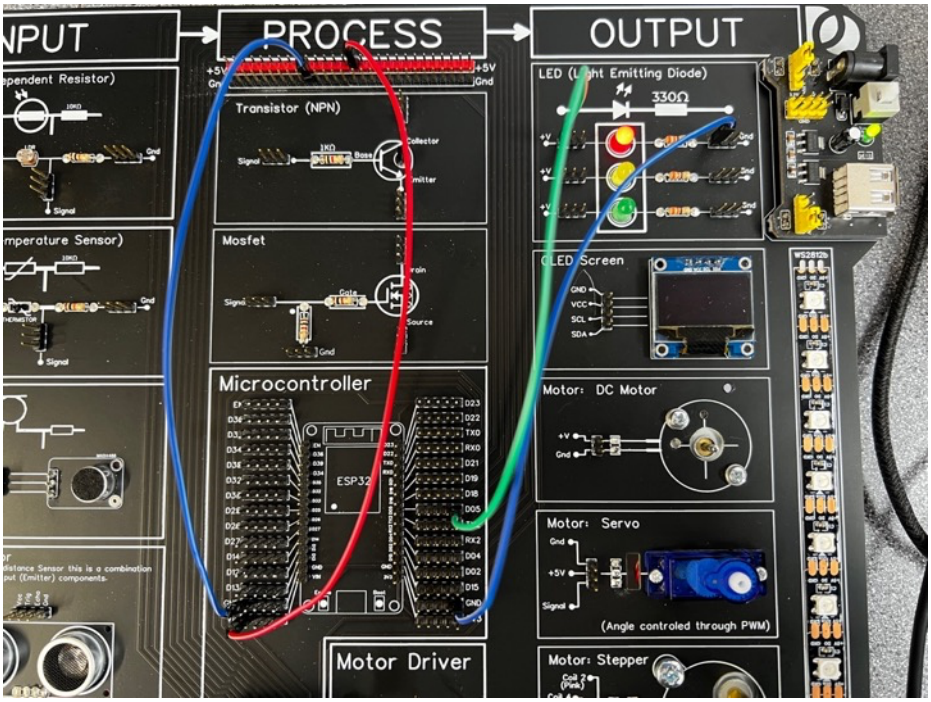
Click the arrow to upload

The LED should now be flashing. You've finished the circuit. WELL DONE!

HELP! If the programme doesn't work, try pressing the small EN button on the ESP32 board to start the programme.

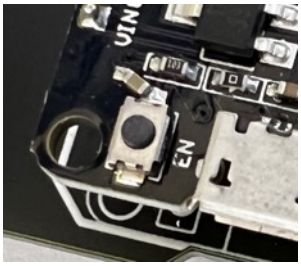
If it still isn't working, go look carefully at your connections.





Challenge 1 - Add a second LED that lights when the first LED goes off

Now think about how you connected and controlled the first LED. Your challenge is to see if you can control a second LED. To do this you need to add to your existing programme and add to your Learn Arduino board circuit.



Think it through and try it out. If you get stuck the solution is in the solutions presentation

No cheating! You wanted to learn, and the best way is to try things out and work through problems

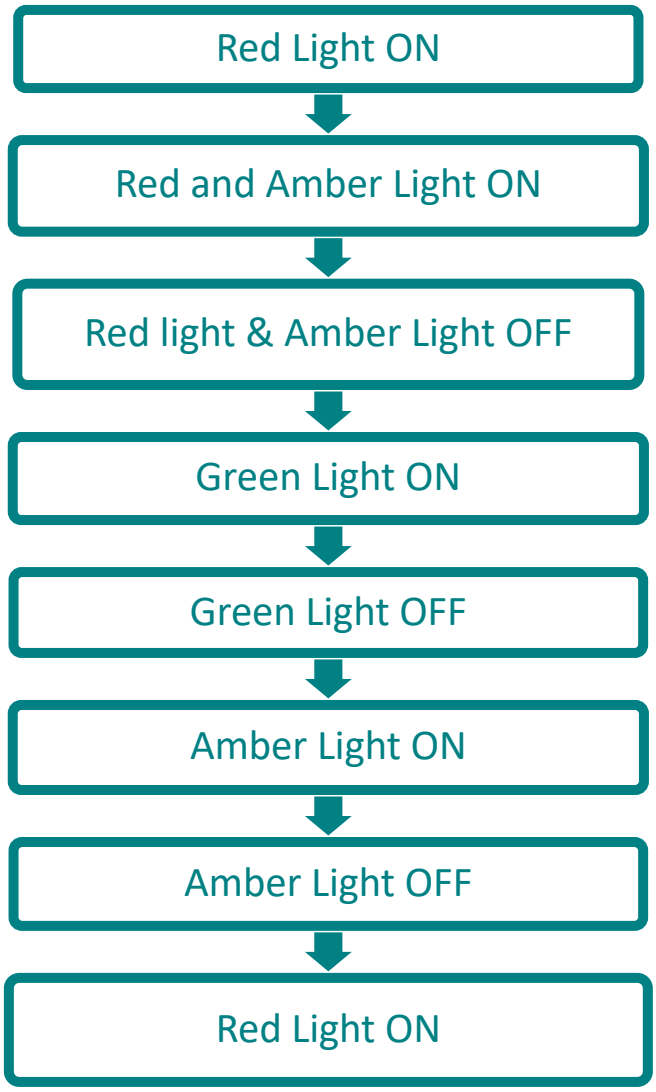
Challenge 2 – Connect the 3rd LED and see if you can design a programme to run a traffic light sequence.

Similar to the previous task, you're now incorporating the last LED and thinking about the sequence. The best way to do this is to write yourself a list of what it has got to do and then fill in the code next to the list.

You can see that the first two commands have been written for you. All you have to do is fill in the rest!

Remember the delays or you might not see the changes.

Refer to the solutions presentation if you are really stuck but remember you need to try it before just looking at the answer!




```
DigitalWrite (5,HIGH);

Delay (1000);

DigitalWrite .......
```

Getting started is always difficult but you've done it!

Copyright Distinctive Objects Ltd 2022